

Day 9 — LEFT JOIN, RIGHT JOIN & FULL JOIN

Week 2 · SQL Querying

Yesterday, you learned INNER JOIN, which returns only matching rows.

Today, you'll learn how to preserve unmatched records. This is essential for real-world data engineering because missing relationships are often meaningful data, not errors.

For a RAG platform, consider these questions:

- Which users have never uploaded a document?
- Which documents have no chunks?
- Which ingestion jobs have no successful embedding?
- Are there documents or records that exist on one side but not the other?

These questions require more than INNER JOIN.

1. Today's Learning Objectives

By the end of Day 9, you should be able to:

- Explain LEFT JOIN
- Explain RIGHT JOIN
- Explain FULL OUTER JOIN
- Distinguish matched and unmatched rows
- Understand when unmatched columns become NULL
- Find users with no documents
- Find documents with no chunks
- Understand why COUNT(*) and COUNT(column) behave differently with LEFT JOIN
- Debug incorrect join conditions
- Choose the appropriate JOIN for a business requirement

Rule: You write the SQL first. I will not provide solutions before your attempt.

2. Database for Today's Exercises

We'll use a slightly expanded RAG schema.

users

	id	name	country	account_status
1	Aarav	India	active	
2	Emma	USA	active	
3	Liam	UK	inactive	
4	Sofia	India	active	
5	Noah	USA	suspended	
6	Olivia	Canada	active	
7	Mia	Germany	active	

documents

	id	user_id	title	file_type	page_count	status
101	1		RAG Fundamentals	pdf	20	processed
102	1		PostgreSQL Guide	pdf	45	processed
103	2		Python Basics	pdf	30	processing
104	3		FastAPI Tutorial	md	15	failed
105	4		Vector Databases	pdf	50	processed
106	5		Docker Guide	pdf	25	processing
107	6		LLM Fundamentals	pdf	60	processed
108	6		Prompt Engineering	md	18	processed
109	2		Embeddings Guide	pdf	55	failed
110	4		RAG Evaluation	pdf	35	processing
111	99		Orphan Document	pdf	10	processed

Notice:

- User 7 has no documents.
- Document 111 references user_id = 99, which does not exist in users.

These two unmatched cases are intentional.

document_chunks

id	document_id	chunk_index	chunk_text
1001	101	0	RAG is a retrieval architecture
1002	101	1	Retrieval improves grounding
1003	102	0	PostgreSQL supports relational queries
1004	103	0	Python variables store values
1005	105	0	Vector databases store embeddings
1006	105	1	Similarity search compares vectors
1007	107	0	LLMs process token sequences

1008	108	0	Prompt design affects output
1009	109	0	Embeddings represent semantic meaning

Notice:

- Document 104 has no chunks.
- Document 106 has no chunks.
- Document 110 has no chunks.
- Document 111 has no chunks.

This will help you understand unmatched rows.

PART 1 — LEFT JOIN

3. What Is a LEFT JOIN?

Syntax:

```
SELECT
  u.name,
  d.title
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id;
```

Meaning:

Return all rows from the left table, and matching rows from the right table.

Here:

LEFT TABLE = users

RIGHT TABLE = documents

Therefore:

- Every user appears.
- Users with documents show their documents.
- Users without documents still appear.
- For users without a matching document, document columns become NULL.

4. INNER JOIN vs LEFT JOIN

Suppose:

users

id	name
1	Aarav
2	Emma
7	Mia

documents

id	user_id	title
101	1	RAG Fundamentals
102	2	Python Basics

INNER JOIN

```
SELECT u.name, d.title
```

```
FROM users u
```

```
INNER JOIN documents d
```

```
ON u.id = d.user_id;
```

Result:

name	title
Aarav	RAG Fundamentals
Emma	Python Basics

Mia disappears.

LEFT JOIN

```
SELECT u.name, d.title
```

```
FROM users u
```

```
LEFT JOIN documents d
```

```
ON u.id = d.user_id;
```

Result:

name	title
Aarav	RAG Fundamentals
Emma	Python Basics
Mia	NULL

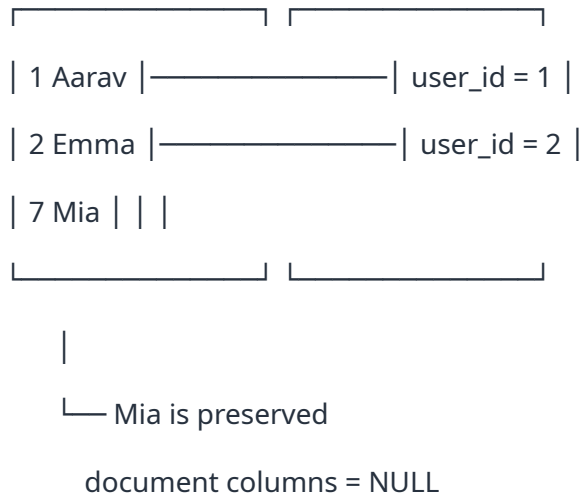
Core difference:

JOIN	Preserves unmatched left rows?
INNER JOIN	No
LEFT JOIN	Yes

5. Visual Mental Model

LEFT JOIN

users documents



Think:

LEFT JOIN protects the rows of the left table.

6. Finding Users Without Documents

This is one of the most important SQL patterns.

Business requirement:

Find users who have never uploaded a document.

Use:

LEFT JOIN

Then identify unmatched rows.

The pattern is:

```
SELECT ...  
FROM users u  
LEFT JOIN documents d  
  ON u.id = d.user_id  
WHERE d.id IS NULL;
```

Why d.id IS NULL?

Because a user with no matching document receives NULL for the document columns.

Result conceptually:

id name

Important pattern

```
LEFT JOIN
```

↓

```
WHERE right_table.primary_key IS NULL
```

↓

unmatched left rows

This is often called an anti-join pattern.

7. Why Check d.id IS NULL?**Suppose you write:**

```
WHERE d.title IS NULL
```

It may work if title is guaranteed non-null, but it is not the clearest choice.

Prefer a column that is guaranteed to identify a real row, usually:

```
WHERE d.id IS NULL
```

This makes your intention explicit:

There is no matching document row.

8. LEFT JOIN + WHERE: Important Trap

Compare these two queries carefully.

Query A

```
SELECT
u.name,

d.title

FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
WHERE d.status = 'processed';
```

Query B

```
SELECT
u.name,

d.title

FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
```

```
AND d.status = 'processed';
```

They are not equivalent.

Query A

The WHERE condition removes rows where d.status is NULL.

Therefore, users with no documents disappear.

This can make the result behave like an INNER JOIN for that condition.

Query B

The condition is part of the matching logic.

All users are preserved, but only processed documents are matched.

This distinction is extremely important in production SQL.

We'll practice it later.

PART 2 — RIGHT JOIN

9. What Is a RIGHT JOIN?

Syntax:

```
SELECT
  u.name,
  d.title
FROM users u
RIGHT JOIN documents d
  ON u.id = d.user_id;
```

Meaning:

Return all rows from the right table, and matching rows from the left table.

Here:

LEFT TABLE = users

RIGHT TABLE = documents

Therefore:

- Every document appears.
- Documents with a matching user show the user.
- Documents without a matching user still appear.

- User columns become NULL for unmatched documents.

10. RIGHT JOIN Example

Using our data:

```
SELECT
  u.name,
  d.id AS document_id,
  d.title
FROM users u
RIGHT JOIN documents d
  ON u.id = d.user_id;
```

The orphan document appears:

name	document_id	title
NULL	111	Orphan Document

Why?

Because document 111 exists in the right table, and RIGHT JOIN preserves right-table rows.

11. RIGHT JOIN Is Usually Rewritable

This:

```
FROM users u
RIGHT JOIN documents d
  ON u.id = d.user_id
```

can be rewritten as:

```
FROM documents d
LEFT JOIN users u
  ON u.id = d.user_id
```

They express the same preservation logic, assuming the same join condition and selected columns.

Engineering recommendation

Many teams prefer LEFT JOIN consistently because it is easier to read:

Start with the entity you want to preserve, then LEFT JOIN related data.

For example:

Need every document?

→ Start FROM documents

→ LEFT JOIN users

You don't need to avoid RIGHT JOIN entirely. Just understand it.

PART 3 — FULL OUTER JOIN

12. What Is FULL OUTER JOIN?

Syntax:

```
SELECT
  u.id AS user_id,
  u.name,
  d.id AS document_id,
  d.title
FROM users u
FULL OUTER JOIN documents d
  ON u.id = d.user_id;
```

Meaning:

Return all matching and unmatched rows from both tables.

This includes:

1. Users with documents
2. Users without documents
3. Documents without users

13. FULL OUTER JOIN Result Conceptually

For a simplified dataset:

users

	id	name
--	----	------

1	Aarav
2	Emma
7	Mia

documents

	id	user_id	title
101	1	RAG Fundamentals	
111	99	Orphan Document	

Result:

user_id	name	document_id	title
1	Aarav	101	RAG Fundamentals
2	Emma	NULL	NULL
7	Mia	NULL	NULL
NULL	NULL	111	Orphan Document

Both unmatched sides survive.

14. FULL JOIN Mental Model

FULL OUTER JOIN

All matching rows

+

Unmatched left rows

+

Unmatched right rows

Or:

LEFT JOIN

+

right-only rows

Conceptually, though PostgreSQL handles this as a join operation rather than you manually combining queries.

15. When Is FULL OUTER JOIN Useful?

Common examples:

Data reconciliation

Compare two systems:

CRM customers



Data warehouse customers

Find:

- Records in both

- Records only in CRM
- Records only in warehouse

Pipeline validation

Compare:

Expected documents



Processed documents

Migration validation

Compare old and new tables.

RAG example

Compare:

documents



embedding_metadata

to identify:

- Documents with embeddings
- Documents without embeddings
- Embedding records without valid documents

PART 4 — NULL AND OUTER JOINS

16. Why Does NULL Appear?

Suppose:

```
SELECT
  u.name,
  d.title
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id;
```

For Mia:

users.id = 7

No document matches:

`documents.user_id = 7`

So PostgreSQL produces:

`u.name = 'Mia'`

`d.title = NULL`

This does not mean the document title is actually stored as NULL.

It means:

There is no matching right-side row.

This distinction matters.

17. COUNT(*) vs COUNT(d.id) with LEFT JOIN

This is a critical engineering concept.

Suppose:

```
SELECT
  u.id,
  u.name,
  COUNT(*) AS row_count,
  COUNT(d.id) AS document_count
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
GROUP BY u.id, u.name;
```

For a user with no documents:

LEFT JOIN still creates one output row

So:

`COUNT(*) = 1`

But:

`d.id = NULL`

Therefore:

`COUNT(d.id) = 0`

Key rule

`COUNT(*)` → counts joined rows

COUNT(d.id) → counts non-NULL document IDs

When counting related records with a LEFT JOIN, COUNT(related_table.id) is usually what you want.

18. Example: Every User and Document Count

Requirement:

Show every user, including users with zero documents.

Use:

```
SELECT
  u.id,
  u.name,
  COUNT(d.id) AS document_count
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
GROUP BY
  u.id,
  u.name;
```

This is a very important real-world pattern.

With INNER JOIN, users with zero documents disappear.

With LEFT JOIN + COUNT(d.id), they remain with count 0.

19. LEFT JOIN + GROUP BY + HAVING

Suppose:

Show every user with at least 2 documents.

You can use:

```
LEFT JOIN
GROUP BY
HAVING COUNT(d.id) >= 2
```

The LEFT JOIN preserves all users initially.

Then HAVING removes users whose count is below 2.

This is a good example of how JOIN and aggregation work together.

PART 5 — JOIN STRATEGY

20. How to Choose the JOIN

Ask:

Requirement: Only users who have documents

INNER JOIN

Requirement: Every user, even those with zero documents

users

LEFT JOIN documents

Requirement: Every document, even those with no valid user

documents

LEFT JOIN users

or equivalently a RIGHT JOIN with users on the left.

Requirement: Everything from both tables

FULL OUTER JOIN

21. Practical RAG Examples

Example A — Users with no uploaded documents

users

LEFT JOIN documents

WHERE documents.id IS NULL

Example B — Documents with no chunks

documents

LEFT JOIN document_chunks

WHERE document_chunks.id IS NULL

Example C — Every document and its owner, including orphan documents

documents

LEFT JOIN users

Example D — Every user and their processed document count, including zero

users

LEFT JOIN documents

ON ... AND status = 'processed'

GROUP BY users

Notice the placement of the status condition. This is a subtle but important point.

PART 6 — DEBUGGING CHALLENGES

Debugging 1 — Missing Users

Developer writes:

```
SELECT
  u.id,
  u.name,
  d.title
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
WHERE d.id IS NULL;
```

Requirement:

Find users who have no documents.

Your tasks

1. Why will this query fail to return the intended users?
2. Which JOIN should be used?
3. Why does d.id IS NULL make sense only with the appropriate JOIN pattern?
4. Rewrite the query.

Debugging 2 — Wrong Preservation Side

Developer writes:

```
SELECT
  u.name,
  d.title
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
WHERE d.id IS NOT NULL;
```

Requirement:

Return every user, including users with no documents.

Your tasks

1. Does this query preserve users without documents in the final result?

2. What does WHERE d.id IS NOT NULL do?
3. How would you write the query if the requirement is truly every user?
4. If the requirement were instead "only users with documents," would INNER JOIN be clearer?

Debugging 3 — COUNT Bug

Developer writes:

```
SELECT
  u.id,

  u.name,

  COUNT(*) AS document_count
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
GROUP BY u.id, u.name;
```

The developer claims:

"This correctly returns 0 for users with no documents."

Your tasks

1. Is that claim correct?
2. Why can COUNT(*) produce 1 for an unmatched left row?
3. Which expression should be used to count actual documents?
4. Rewrite the query.

Debugging 4 — Filter Placement

Developer wants:

Return every user, along with the number of processed documents. Users with no processed documents should show 0.

They write:

```
SELECT
  u.id,

  u.name,

  COUNT(d.id) AS processed_documents
FROM users u
LEFT JOIN documents d
  ON u.id = d.user_id
```

```
WHERE d.status = 'processed'
```

```
GROUP BY u.id, u.name;
```

Your tasks

1. Why might users with no processed documents disappear?
2. Where should the status condition go to preserve every user?
3. Rewrite the query.
4. Explain the difference between filtering in ON and filtering in WHERE.

PART 7 — SQL EXERCISES

Use the provided users, documents, and document_chunks tables.

Basic

Q1

Write a query using LEFT JOIN to return:

user_name

document_title

for every user, including users with no documents.

Q2

Find users who have no documents.

Return:

user_id

user_name

Q3

Write a query using RIGHT JOIN to return every document and its owner's name.

Include orphan documents.

Return:

document_id

document_title

user_name

Q4

Rewrite Q3 using LEFT JOIN instead of RIGHT JOIN.

The result should represent the same business requirement.

Q5

Write a FULL OUTER JOIN query that returns:

user_id

user_name

document_id

document_title

Include unmatched users and unmatched documents.

Intermediate

Q6

Return every user and their document count, including users with zero documents.

Output:

user_id

user_name

document_count

Q7

Return every document and its chunk count, including documents with zero chunks.

Output:

document_id

document_title

chunk_count

Hint: Think about COUNT(c.id).

Q8

Find documents that have no chunks.

Return:

document_id

document_title

Q9

Find users who have at least 2 documents, including only users who actually meet that condition.

Return:

user_id

user_name

document_count

Use:

LEFT JOIN

GROUP BY

HAVING

Q10

Return every user and the number of processed documents they own.

Users with no processed documents must remain in the result with count 0.

Output:

user_id

user_name

processed_documents

This tests filter placement.

Advanced

Q11

Find users who have zero processed documents, including:

- Users with only failed documents
- Users with only processing documents
- Users with no documents at all

Return:

user_id

user_name

Q12

Find all documents whose owner does not exist in users.

Return:

document_id

document_title

user_id

Use a LEFT JOIN anti-join pattern.

Q13

Return every document with:

document_id

document_title

owner_name

chunk_count

Requirements:

- Include orphan documents.
- Include documents with zero chunks.
- Use appropriate JOINS.
- Group correctly.

Think carefully about joining all three tables.

Q14

Return every user with:

user_id

user_name

total_documents

processed_documents

failed_documents

Requirements:

- Include users with zero documents.

- Use LEFT JOIN.
- Use conditional aggregation.
- Group by the user.

PART 8 — PRACTICAL ENGINEERING PROBLEMS

Problem 1 — RAG Coverage Audit

Your team wants to identify users who have registered but never uploaded a document.

Requirement

Return:

user_id

user_name

country

Include only users with no documents.

Constraints

- Use LEFT JOIN.
- Use IS NULL.
- Do not use multiple queries.

Problem 2 — Chunking Pipeline Audit

Your RAG ingestion pipeline has uploaded documents, but some have not yet been chunked.

Requirement

Find all documents with zero chunks.

Return:

document_id

document_title

document_status

Constraints

- Use LEFT JOIN.
- Join documents to document_chunks.
- Use the appropriate NULL check.

Problem 3 — Complete User Dashboard

Build a query for a dashboard that shows every user, even users with no documents.